

Bitácora de decisiones técnicas FlashSale

1. Dividir el sistema en 4 proyectos

Decidí separar el proyecto en 4 partes: Shared (lo que comparten todos), Generator (genera los eventos), Consumer (los procesa) y Api (los expone). Pensé en hacerlo todo en un solo proyecto, pero así cada parte hace una sola cosa, es más fácil de explicar y puedo escalar cada una por separado, por ejemplo correr varios Consumers sin tocar la Api.

2. Una interfaz compartida (IBalanceRepository)

En Shared solo dejé el contrato, es decir qué operaciones existen, y cada proyecto que lo necesita hace su propia implementación con Mongo. Pude haber metido el código de Mongo directamente en Shared, pero si en algún momento cambio de base de datos no tendría que tocar todo el sistema, solo esa implementación.

3. Particiones de Kafka usando el ProductId como key

Usé un solo topic (purchase_intents) con 4 particiones, y el ProductId como key de cada mensaje. Consideré usar un topic por cada producto o mandar los mensajes sin ninguna key, pero con la key todos los eventos de un mismo producto siempre caen en la misma partición y quedan en orden, mientras que productos distintos se reparten entre las 4 particiones y se procesan en paralelo. Esto es el sharding que pedía el proyecto.

4. 4 particiones

Dejé el topic fijo en 4 particiones. Con una sola no habría paralelismo real, y con un número mucho más alto estaría complicando de más la configuración para un solo broker de prueba. Con 4 puedo correr hasta 4 Consumers en paralelo, que es suficiente para este proyecto.

5. Confirmar los mensajes de Kafka a mano, no automático

Desactivé el auto-commit y solo confirmo un mensaje después de haberlo guardado bien en Mongo. Si dejaba el auto-commit por defecto, y el programa se caía justo después de leer un mensaje pero antes de guardarlo, ese mensaje se perdería. Confirmando a mano, Kafka me lo vuelve a mandar si algo falló a medias.

6. Qué hacer con mensajes malformados

Si un mensaje no se puede leer bien o le faltan datos, lo descarto y sigo, dejando un aviso en el log, en vez de dejar que el programa se caiga por un solo mensaje malo. En un flujo real siempre va a haber algún dato corrupto o incompleto, y no tiene sentido parar todo el sistema por eso.

7. Evitar procesar el mismo evento dos veces

Guardo en Mongo, en una colección aparte, el ID de cada evento ya procesado, y lo reviso antes de aplicar uno nuevo. Pensé en llevar ese control solo en memoria, con una lista dentro del programa, pero si el programa se reinicia esa lista se pierde. Guardándolo en Mongo, el control de duplicados sobrevive a un reinicio.

8. MongoDB en vez de una base de datos relacional

Elegí MongoDB para guardar el balance de stock en vez de SQL Server o PostgreSQL. Los datos no tienen un esquema tan rígido y lo que más hago es actualizar un documento por producto (upsert), así que no necesito relaciones complejas entre tablas.

9. El balance se calcula al leer, no se guarda fijo

BalanceActual y Estado (Normal, Riesgo, Sobreventa) se calculan cada vez que se consultan, no los guardo como un valor fijo en la base. Así, si más adelante cambio la lógica de qué es 'riesgo' o 'sobreventa', no tengo que salir a actualizar todos los documentos viejos.

10. Catálogo de productos con popularidad distinta

Le di a cada producto un peso distinto a la hora de generar eventos al azar, para que unos salgan mucho más que otros, en vez de elegir el producto totalmente al azar con la misma probabilidad para todos. En una venta flash real no todos los productos se piden igual, unos se vuelven virales y otros casi nadie los pide, así que la prueba se ve más real.

11. Envío individual contra envío por lote

Para un solo evento espero la confirmación de Kafka antes de seguir. Para el lote masivo, mando todo de una vez y confirmo al final, en vez de usar el mismo método uno por uno también para los lotes grandes. Esperar la confirmación mensaje por mensaje en un lote de miles sería muy lento, y mandarlos de corrido es justo lo que necesito para medir el throughput.

12. Kafka y Mongo con Docker, en modo KRaft sin Zookeeper

Levanté Kafka y Mongo con docker-compose, usando Kafka en su modo nuevo (KRaft) que no necesita Zookeeper, en vez de instalar todo directo en Windows o usar Kafka con Zookeeper clásico. Con Docker el entorno queda igual en cualquier máquina y me evito líos de instalación, y KRaft es más simple porque es un contenedor menos. De hecho tuve un problema real: tenía MongoDB instalado directo en Windows que chocaba con el puerto del Mongo de Docker, y lo resolví apagando ese servicio de Windows.

13. Dashboard simple en HTML, sin usar React ni nada parecido

Hice una sola página HTML con JavaScript que consulta la Api y se refresca sola cada 5 segundos, en vez de armar un proyecto de frontend aparte con React. El proyecto solo pide un tablero que muestre el balance, no una tecnología específica, así que una página simple cumple lo mismo con mucho menos trabajo y es más fácil de explicar en la defensa.